

Atlas — System Design Document (HLD & LLD)

Product: Atlas — an AI learning memory that captures, understands, and remembers everything a person learns online. **This document:** the High-Level Design (HLD) and Low-Level Design (LLD) for the ingestion → intelligence → retrieval system.

Field	Value
Version	0.1 (draft)
Status	For review
Scope	Full-system design, but Starter (Phase 0) is the extension/ folder only
Starter deliverable	The extension captures the defined sources and writes each raw artifact to S3. Nothing else.
Stack — starter	Extension: MV3 · WXT · React · TypeScript · a thin capture endpoint (FastAPI) whose only job is to accept/sign the upload to S3
Stack — later phases	Full FastAPI backend, Next.js web app, Postgres + pgvector, Redis workers, LLM pipeline
Out of scope for starter	Pipeline, summaries, chunks, concepts, chat, coverage, the web app, and Postgres — all Phase 1+

1. Introduction

1.1 Purpose

Define the architecture and implementation contract for Atlas so that engineering can build the POC without ambiguity. The HLD fixes the components, boundaries, and principles; the LLD fixes the schemas, interfaces, algorithms, and coding standards.

1.2 Audience

Backend, frontend, and infra engineers; reviewers evaluating the design before build.

1.3 Definitions

Term	Meaning
Space	A Learning Space — a goal-scoped container ("System Design"). All sources, concepts, and state are scoped to a space.
Source	One captured artifact: a video, article, AI chat, PDF, or note.
Raw artifact	The immutable captured payload (transcript, page text, chat dump) stored in S3.
Chunk	A retrieval-sized slice of a source's text with an embedding.
Concept	A canonical, de-duplicated unit of knowledge within a space.
Capture	The synchronous act of saving a source.
Pipeline	The asynchronous processing that turns a raw artifact into summaries, chunks, and concepts.

2. Product Overview

2.1 Goals

- One-click, zero-friction capture from the browser into an active Space.
- Turn every captured source into an instant summary and a searchable, connected knowledge base.
- Model what the user *knows* (coverage, gaps, "you already know 70% of this").
- Make capture feel free: the user's click returns instantly; intelligence is computed in the background.

2.2 Non-goals (this revision)

- No mobile client.
- No verbatim resurfacing of copyrighted source text (summaries, highlights, and links only).
- No quiz/flashcard scheduling engine yet (schema leaves room for it).

2.3 Guiding product constraint

Capture is synchronous and fast; intelligence is asynchronous. Every design decision optimizes the felt latency of the save.

2.4 Starter scope (Phase 0) — the only thing we build first

The starter is intentionally tiny: the `extension/` folder captures a source and stores its raw artifact in S3. No summarizing, no Postgres, no web app. Everything downstream in this document (§4.6 onward) is deliberately deferred so the roadmap stays visible without being in scope.

What the extension captures per source type:

Source type	Captured detail	Raw artifact written to S3
YouTube video	title, url, channel/author, duration, full transcript (with timestamps)	<code>raw/transcript.json</code>
Web article	title, url, author, cleaned article text (Readability)	<code>raw/page.html</code> + <code>raw/extracted.txt</code>
AI chat	title, url, full message thread (role + text)	<code>raw/chat.json</code>
PDF	title, url/filename, the file itself	<code>original.pdf</code> (pre-signed PUT)

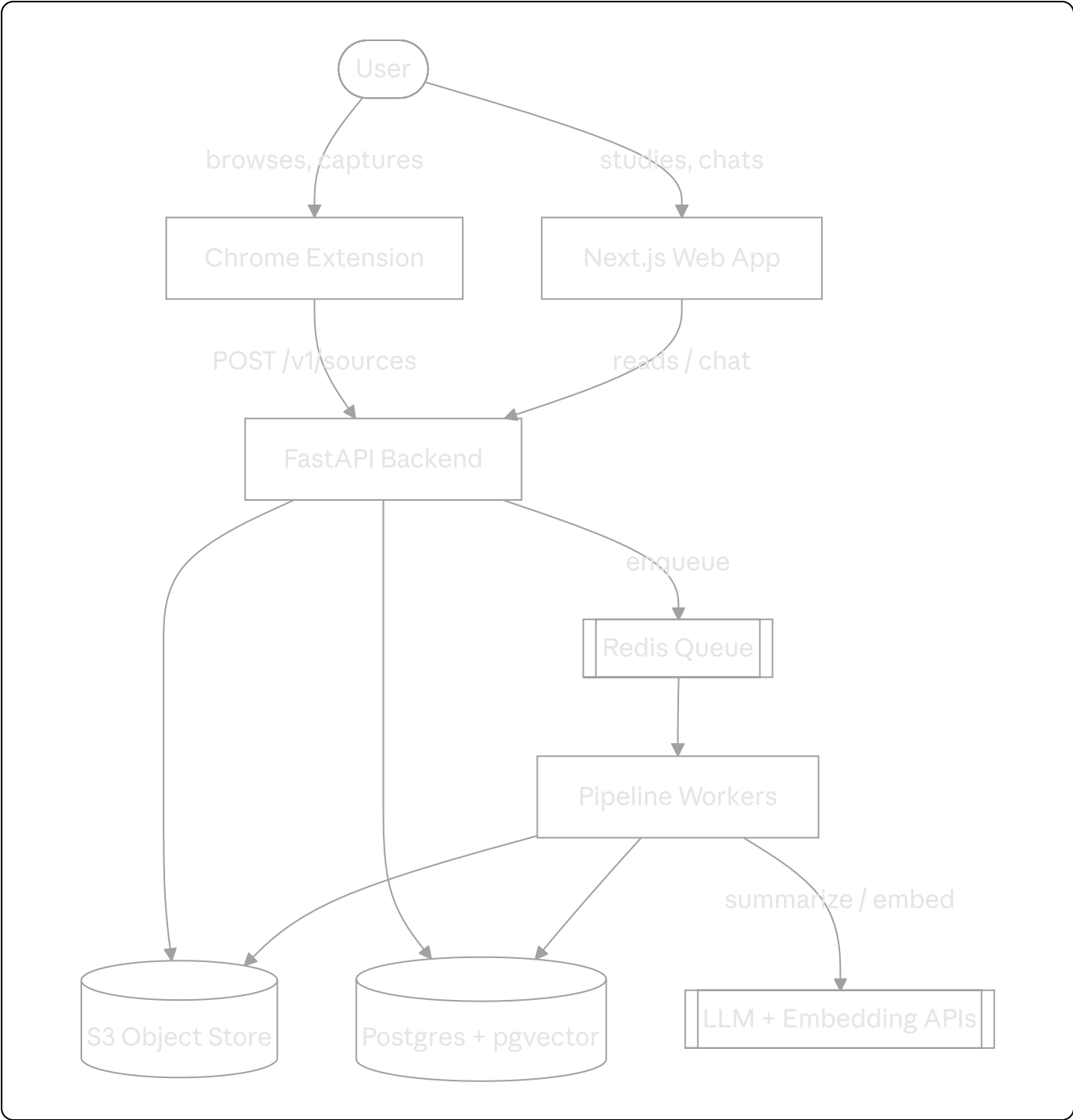
Common metadata on every capture: `user_id` (from auth), `space_id` (active space), `type`, `captured_at`, `content_hash`.

Starter definition of done: from a supported page, the auto-capture card saves the source, the raw artifact lands in S3 under the user's prefix, and the extension shows a "Saved ✓" confirmation. That is the whole Phase 0.

Phasing note: §3 (HLD) and §4.2–§4.9 describe the full target system. For the starter, only §3.1–§3.4 (context/principles), §4.1 (`extension/` folder), §4.3 (S3 layout), §4.4 (auth), and the capture slice of §4.5 apply. The rest is Phase 1+.

3. High-Level Design (HLD)

3.1 System context



3.2 Components and responsibilities

Component	Tech	Responsibility
Extension	MV3, WXT, React, TS	Detect capturable content, auto-surface capture card, send captures. Holds no business logic beyond capture. Background worker is the only auth holder.

Component	Tech	Responsibility
Web app	Next.js (App Router)	Browse spaces, read summaries, chat with a space, coverage/gaps. Reads derived data only.
API (ingestion + read)	FastAPI	Auth, validation, fast capture ack, raw persistence, job enqueue, read APIs, RAG orchestration. Stateless.
Object store	S3 / R2 / Supabase Storage	Immutable raw artifacts, keyed per user. Source of truth for re-processing.
Relational store	Postgres + pgvector	All structured, queryable data + vector index. The frontend's read source.
Queue	Redis + async worker (Arq)	Durable hand-off from sync capture to async processing.
Pipeline workers	Python (shares <code>app</code> package)	chunk → embed → summarize → extract → resolve → update state. Idempotent, retryable.
LLM providers	Anthropic Claude + embedding model	Summaries/chat/quiz (Claude); extraction/tagging (cheap model); vectors (embedding model).

3.3 Architectural principles

- Separation of concerns by layer.** Router → Service → Repository → Model. No business logic in routers or SQL in services.
- Sync capture / async intelligence.** The capture endpoint never calls an LLM. It persists + enqueues + returns `202`.
- Two-store split.** Blobs in S3 (immutable, cheap, replayable); structured data in Postgres (queryable). Never store large text in Postgres rows; never query S3 on the hot path.
- Stateless API.** No in-memory session state; horizontal scale by default. State lives in Postgres/Redis/S3.
- Idempotent, replayable workers.** Every pipeline stage can re-run safely, keyed by `source_id` + `content_hash`.
- One brain in the extension.** Only the background worker touches auth/API; content scripts are dumb UI.
- Least privilege.** Clients never hold S3 credentials; access is via short-lived pre-signed URLs scoped to the user prefix.

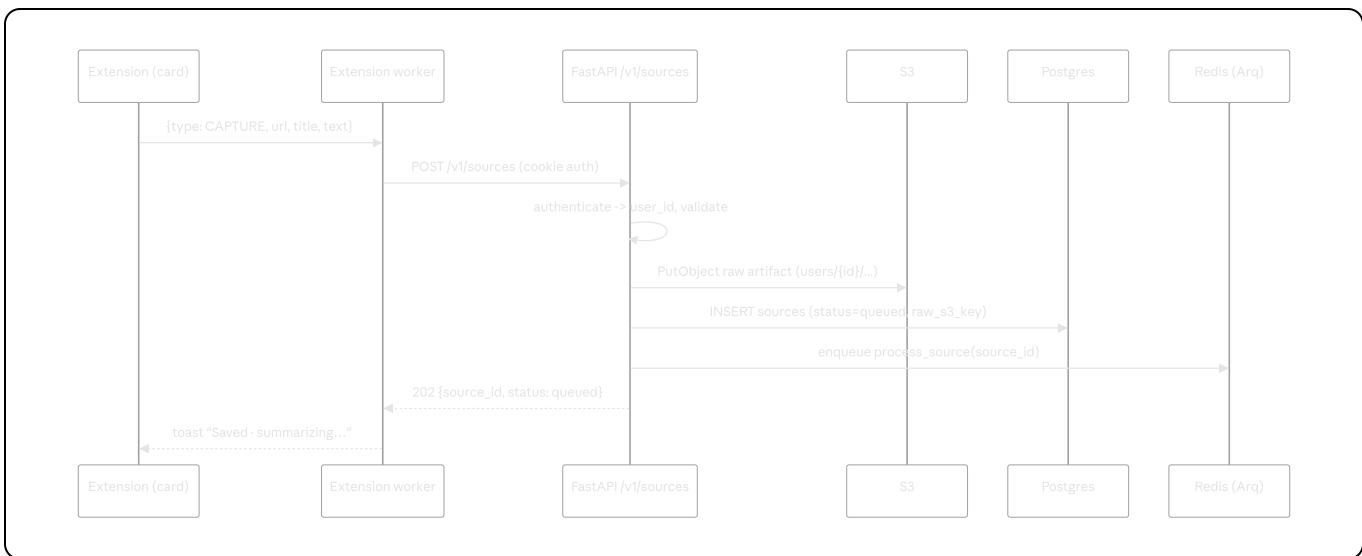
8. **Fail visibly, degrade gracefully.** Processing failures surface as a source status, never a lost capture.

3.4 Technology choices and rationale

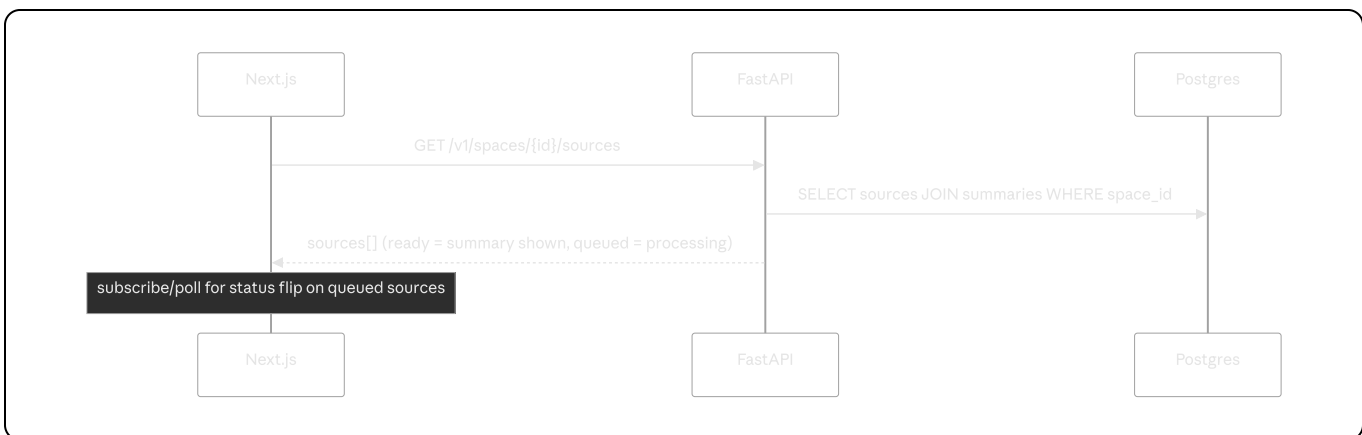
Concern	Choice	Why	Alternatives considered
Backend framework	FastAPI	Async, Pydantic-native validation, first-class OpenAPI, great DI.	Django (heavier), Flask (less async/typing).
ORM / migrations	SQLAlchemy 2.0 (async) + Alembic	Typed, async, mature migrations.	SQLModel (thin, less control), raw asyncpg.
Validation	Pydantic v2	Shared request/response contracts, fast.	marshmallow.
Vector store	pgvector in Postgres	One datastore; HNSW is enough at POC scale.	Qdrant/Pinecone (add only at strain).
Job queue	Arq (Redis)	Async-native, pairs with FastAPI's event loop, minimal infra.	Celery (mature, heavier), Dramatiq, Inngest (JS-first).
Object storage	S3-compatible (R2 recommended)	S3 API, zero egress on R2.	AWS S3, Supabase Storage (also S3-compatible).
Frontend	Next.js App Router + TS	Server components for fast reads, RSC data fetching.	Remix, SPA.
Client data cache	TanStack Query	Status polling, cache, retries.	SWR.
LLM	Anthropic Claude (+ cheap tier)	Strong synthesis for user-facing output; cheap model for extraction.	Model split is a cost lever, not a single choice.

3.5 Data flows

Write path (capture):



Read path (open a space):



3.6 Cross-cutting concerns (summary; details in LLD)

- **Security:** cookie auth (shared with extension), CSRF on state-changing web calls, rate limiting, strict input validation, S3 least-privilege.
- **Scalability:** stateless API + worker pool scale independently; queue absorbs capture spikes.
- **Reliability:** durable queue, idempotent stages, per-stage retries, dead-letter to `failed` status.
- **Observability:** structured JSON logs with request/trace IDs, metrics, error tracking.
- **Cost:** model split, embedding cache by `content_hash`, batch embeds.
- **Privacy/Copyright:** raw text is internal-only; frontend surfaces summaries + user highlights + links.

4. Low-Level Design (LLD)

Starter (Phase 0) covers only: §4.1 (`extension/` folder), §4.3 (S3 layout), §4.4

(auth/session), and the **capture slice** of §4.5. Sections §4.2 (Postgres data model), §4.6 (pipeline), §4.7 (concepts), §4.8 (chat/RAG), and §4.9 (frontend) are **Phase 1+** and are retained here as the forward design, not starter work.

4.1 Repository structure

The starter lives entirely in the `extension/` folder. The backend is a **separate folder/service** and is referenced here only as the endpoint the extension calls — its internal structure is out of scope for the starter and intentionally not detailed in this document.


```

atlas/
├ extension/                                # ← Phase 0: the only folder we build now
│   ├── wxt.config.ts                       # MV3 manifest, permissions,
host_permissions
│   ├── src/
│       ├── background/                     # the "brain": the ONLY holder of auth +
S3/API calls
│           ├── index.ts                   # service-worker entry; message router
│           ├── session.ts                 # reads /v1/session, caches active space
(chrome.storage.session)
│           └── capture.ts                 # builds payload, calls capture endpoint,
uploads to S3
│       ├── content/                       # per-site content scripts (dumb UI; no
auth)
│           ├── youtube/                   # transcript + metadata extraction
│           ├── article/                   # Readability extraction
│           ├── ai-chat/                   # thread extraction
│           └── capture-card/              # the auto-surfacing capture card (React)
│       ├── popup/                         # space switcher UI (React)
│       ├── lib/
│           ├── api.ts                     # typed client for the capture endpoint
│           ├── extractors/                # pure functions: DOM -> normalized
capture payload
│           └── types.ts                    # capture payload types (generated from
backend OpenAPI)
│       └── messaging.ts                    # typed content+background message
contracts
│   ├── tests/                             # vitest (extractors) + Playwright
(capture flow)
│   └── package.json
│
└ backend/                                 # SEPARATE service – NOT detailed here
(starter uses only its capture endpoint)
└ frontend/                               # SEPARATE service – Phase 1+

```

Extension architecture rules (enforced):

- **One brain.** Only `background/` holds the session and talks to the network. `content/` and `popup/` are dumb UI that send typed messages to the background worker.
- **Extractors are pure.** DOM → payload transforms in `lib/extractors/` take input and return a payload with no side effects, so they are unit-testable without a browser.
- **No secrets in content scripts.** Content scripts share the page's world; they never see the session token or S3 credentials.

- **Types come from the backend.** The capture payload type is generated from the backend's OpenAPI schema so the extension and endpoint never drift.

4.2 Data model (*Phase 1+*)

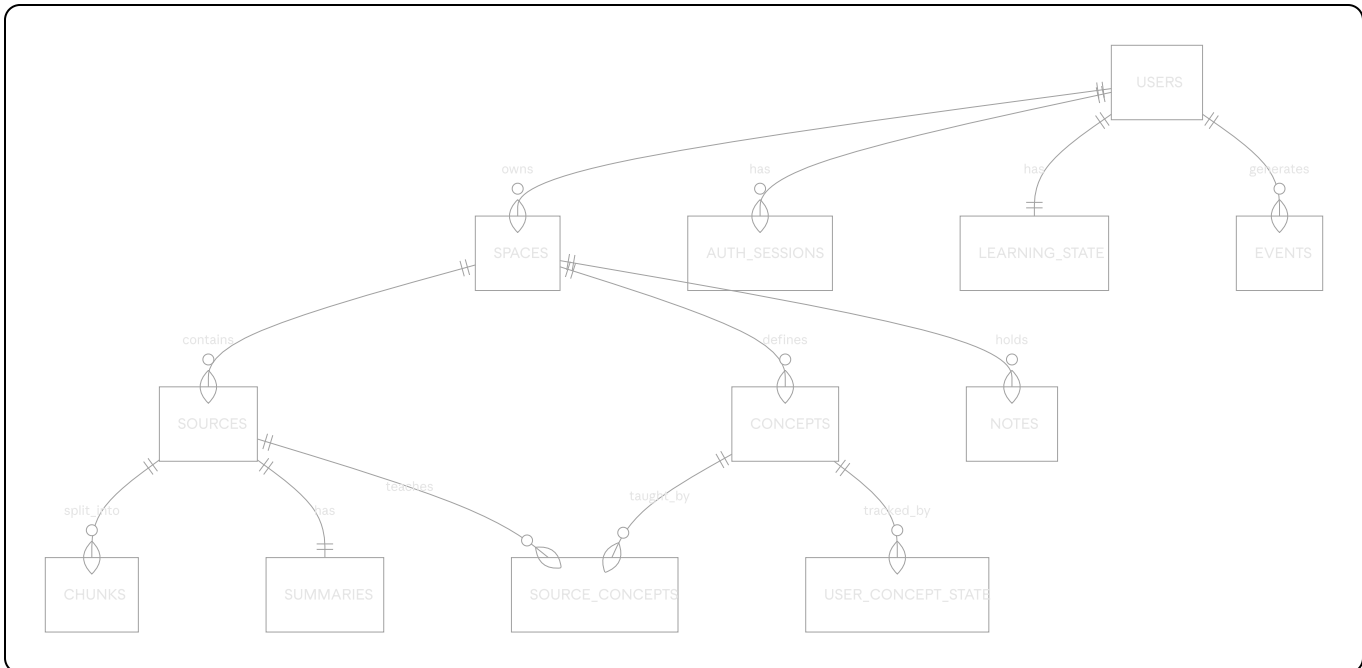


Table specifications (Postgres; `uuid` PKs via `gen_random_uuid()`; all timestamps `timestampz`):

Table	Key columns	Notes / indexes
users	id, email UNIQUE, auth_provider, created_at	
auth_sessions	id, user_id FK, token_hash, expires_at, created_at	token_hash = SHA-256 of opaque cookie value. Index (token_hash).
learning_sessions	user_id PK/FK, active_space_id FK NULL, started_at, last_active_at	One active space per user; read by extension.
spaces	id, user_id FK, name, goal_text, coverage_pct (cache), created_at	Index (user_id).
sources	id, space_id FK, user_id FK, type enum, url, title, author, raw_s3_key, content_hash, processing_status enum, error, captured_at, processed_at	Index (space_id), (user_id, processing_status), (content_hash).
chunks	id, source_id FK, position, text, token_count, anchor jsonb, embedding vector(1536)	HNSW index on embedding (vector_cosine_ops). Index (source_id).
summaries	source_id PK/FK, bullets jsonb, key_concepts jsonb, examples jsonb, interview_points jsonb, model, created_at	1:1 with source.
concepts	id, space_id FK, canonical_name, definition, embedding vector(1536), created_at	HNSW on embedding. Unique (space_id, canonical_name).
source_concepts	source_id FK, concept_id FK, salience float	PK (source_id, concept_id). Index (concept_id).
user_concept_state	user_id FK, concept_id FK, exposure_score, mastery_score, last_reviewed_at, next_review_at	PK (user_id, concept_id). Index (user_id).
notes	id, space_id FK, source_id FK NULL, user_id FK, body, kind	

Table	Key columns	Notes / indexes
	enum(<u>note</u> , <u>highlight</u>), <u>anchor</u> jsonb, <u>created_at</u>	
<u>events</u>	<u>id</u> , <u>user_id</u> FK, <u>space_id</u> FK, <u>type</u> , <u>payload</u> jsonb, <u>occurred_at</u>	Powers timeline / "resume". Index (<u>user_id</u> , <u>occurred_at</u>).

Enums: source_type = {youtube, article, ai_chat, pdf, note}; processing_status = {queued, fetching, chunking, embedding, summarizing, extracting, ready, failed}.

4.3 Object storage layout

```
s3://atlas-artifacts/
  users/{user_id}/
    spaces/{space_id}/
      sources/{source_id}/
        raw/transcript.json      # youtube
        raw/page.html           # article
        raw/extracted.txt        # normalized text
        raw/chat.json            # ai chat
        original.pdf             # pdf (presigned upload)
```

- **Per-user prefix first** → IAM/pre-signed scoping to users/{id}/*, one-shot GDPR deletion, natural sharding.
- **Small text payloads** (transcript/article) are sent through the API (PutObject). **Large files** (PDF) use a pre-signed PUT.
- **Reads** are always via short-lived pre-signed GET minted by the API; clients never hold credentials.

4.4 Auth & session design

- On web login, the API issues an **opaque session token** in a cookie: Set-Cookie: atlas_session=<opaque>; Domain=.atlas.ai; Secure; HttpOnly; SameSite=None; Max-Age=... The server stores only token_hash in auth_sessions.
- **Extension handshake:** host_permissions: ["https://*.atlas.ai/*"]. The background worker calls the API with credentials: "include"; the browser attaches the cookie. GET /v1/session returns { user, active_space }. 401 → extension shows a signed-out state.
- **Active space** is server state (learning_sessions.active_space_id), set by POST /v1/session/active from either web ("Start learning") or the popup.

- **CSRF:** state-changing requests from the web app carry a double-submit CSRF token; the extension's cross-origin calls are authenticated by cookie + `Origin` allow-listing.

4.5 API specification (v1)

Base: `/v1`. Auth: `atlas_session` cookie. Errors use a uniform envelope.

Starter endpoints: only `GET /session`, `POST /session/active`, `POST /sources`, and `POST /sources/upload-url` are needed for Phase 0. In the starter, `POST /sources` **only writes the raw artifact to S3 and returns** — no Postgres row, no queue, no enqueue. The DB-insert + `enqueue` shown in the service snippet below is the Phase 1+ form; the starter version stops after the S3 write. All other endpoints are Phase 1+.

Method	Path	Purpose	Success	Phase
GET	<code>/session</code>	Current user + active space	<code>200</code>	0
POST	<code>/session/active</code>	Set active space <code>{space_id}</code>	<code>204</code>	0
POST	<code>/sources</code>	Capture a source (starter: write raw → S3)	<code>202</code>	0
POST	<code>/sources/upload-url</code>	Pre-signed PUT for PDFs	<code>200</code>	0
POST	<code>/spaces</code>	Create space <code>{name, goal_text}</code>	<code>201</code>	1+
GET	<code>/spaces</code>	List user's spaces	<code>200</code>	1+
GET	<code>/spaces/{id}/sources</code>	Sources + summaries (+ status)	<code>200</code>	1+
GET	<code>/spaces/{id}/coverage</code>	Coverage / gap report	<code>200</code>	1+
POST	<code>/spaces/{id}/chat</code>	RAG chat <code>{query}</code> → answer + citations	<code>200</code>	1+
GET	<code>/sources/{id}</code>	Source detail + status	<code>200</code>	1+
GET	<code>/sources/{id}/original</code>	Pre-signed GET to raw	<code>200</code>	1+
POST	<code>/notes</code>	Create note/highlight	<code>201</code>	1+

Error envelope:

```
json
```

```
{ "error": { "code": "SPACE_NOT_FOUND", "message": "No space with that id", "de
```

Codes map to HTTP: validation → (422), auth → (401), forbidden → (403), not found → (404), conflict → (409), rate limit → (429), unexpected → (500).

Capture contract ((POST /v1/sources)):

```
python

# domains/sources/schemas.py
class CaptureRequest(BaseModel):
    space_id: UUID
    type: SourceType
    url: HttpUrl | None = None
    title: str = Field(min_length=1, max_length=500)
    author: str | None = None
    content: str | None = None          # transcript/article text (small payloads)
    anchor: dict | None = None         # optional timestamp-selector metadata

class CaptureResponse(BaseModel):
    source_id: UUID
    processing_status: ProcessingStatus # "queued"
```

Router (thin; no logic):

```
python

# domains/sources/router.py
@router.post("", status_code=202, response_model=CaptureResponse)
async def capture_source(
    body: CaptureRequest,
    user: User = Depends(get_current_user),
    svc: SourceService = Depends(get_source_service),
) -> CaptureResponse:
    return await svc.capture(user=user, payload=body)
```

Service (orchestration + transaction + enqueue):

```
python
```

```
# domains/sources/service.py
async def capture(self, *, user: User, payload: CaptureRequest) -> CaptureResponse:
    await self.spaces.assert_owned(payload.space_id, user.id) # authz
    content_hash = sha256(payload.content or payload.url)
    key = build_raw_key(user.id, payload.space_id, source_id := uuid4(), payload.content)
    await self.storage.put_text(key, payload.content or "") # S3
    source = await self.repo.insert( # PostgreSQL
        id=source_id, user_id=user.id, space_id=payload.space_id,
        type=payload.type, url=payload.url, title=payload.title,
        raw_s3_key=key, content_hash=content_hash,
        processing_status=ProcessingStatus.QUEUED,
    )
    await self.queue.enqueue("process_source", source.id) # Arq
    return CaptureResponse(source_id=source.id, processing_status=source.processing_status)
```

4.6 Async pipeline (*Phase 1+*)

State machine (`processing_status`):

```
queued → fetching → chunking → embedding → summarizing → extracting → ready
      |_____| (any) _____|→ failed
```

Task entrypoint (Arq):

```
python
```

```
# workers/pipeline/tasks.py
async def process_source(ctx, source_id: UUID) -> None:
    src = await sources_repo.get(source_id)
    if src.processing_status == ProcessingStatus.READY:
        return # idempotent no-op
    raw = await storage.get_text(src.raw_s3_key)
    text = normalize(src.type, raw) # strip chrome / join
    await sources_repo.set_status(source_id, "chunking")
    chunks = chunk(text, target_tokens=800, overlap=100, anchors=src.anchor)
    await sources_repo.set_status(source_id, "embedding")
    vectors = await embeddings.embed_batch([c.text for c in chunks]) # cached
    await chunks_repo.bulk_upsert(source_id, chunks, vectors)
    await sources_repo.set_status(source_id, "summarizing")
    summary = await llm.summarize(text, model="claude") # user-fac
    await summaries_repo.upsert(source_id, summary)
    await sources_repo.set_status(source_id, "extracting")
    candidates = await llm.extract_concepts(text, model="cheap") # high-vol
    await resolve_concepts(src.space_id, candidates) # §4.7
    await state_repo.bump_exposure(src.user_id, src.space_id)
    await events_repo.add(src.user_id, src.space_id, "source_ready", {"source_id": source_id})
    await sources_repo.set_status(source_id, "ready")
```

Reliability:

- **Idempotency:** re-entry checks status; `(chunks)`/`(summaries)` upsert on `(source_id, position)` / `(source_id)`; embeddings cached by content hash.
- **Retries:** Arq per-task retry with exponential backoff; each stage is safe to replay.
- **Poison handling:** after max retries → `(failed)` + `(error)` message; the source still exists (capture is never lost) and can be manually re-queued.
- **Concurrency:** worker pool sized to embedding/LLM rate limits; a per-space advisory lock avoids racing concept resolution.

4.7 Concept extraction & resolution (*Phase 1+*)

The high-value, correctness-critical step. Prevents "Cache" / "Caching" / "caching strategies" from fragmenting and breaking coverage math.

Thresholds are configurable and tuned against a labeled set. All writes for one source run under the per-space advisory lock so two concurrent sources cannot create duplicate concepts.

4.8 RAG chat flow (*Phase 1+*)

1. Authorize space ownership.
2. Embed the query.
3. `SELECT ... FROM chunks WHERE source_id IN (space sources) ORDER BY embedding <=> :q LIMIT k` (pgvector cosine, HNSW).
4. Build a grounded prompt with the top-k chunks + their `anchor` metadata.
5. Call Claude with instructions to cite each claim to `source_id` + timestamp/selector.
6. Return `{ answer, citations[] }`; the frontend renders citations as deep links.

4.9 Frontend design (*Phase 1+*)

Routes (App Router):

```
/                # landing / spaces list
/spaces/[id]      # space overview: sources + summaries
/spaces/[id]/chat # ask-your-memory
/spaces/[id]/coverage # coverage & gaps
/sources/[id]     # source detail + original
```

- **Reads** use Server Components hitting the API directly (fast first paint of precomputed summaries).
- **Live status:** `queued` sources render a "processing" card; a client hook (TanStack Query) polls `GET /v1/sources/{id}` (or subscribes via Realtime) and flips to the

summary when `ready` — the web-side mirror of the extension's toast.

- **Types** are generated from the FastAPI OpenAPI schema into `packages/shared-types`, so web + extension consume the same contracts.
 - **No secrets client-side**; all privileged calls go through the API.
-

5. Coding standards & engineering practices

5.1 Python / FastAPI

- **Python 3.12**, full type hints; **mypy** in strict mode is a CI gate.
- **Ruff** for lint + format (single tool); import sorting on.
- **Pydantic v2** for all external I/O; never accept unvalidated `dict` at a boundary.
- **Layered architecture** (router/service/repository/model); no business logic in routers, no ORM in services.
- **Async everywhere** on I/O paths (SQLAlchemy async, aioboto3, httpx).
- **Custom exception hierarchy** (`AppError` → `NotFoundError`, `AuthError`, ...) with FastAPI exception handlers mapping to the error envelope. No bare `except`.
- **Config** via `pydantic-settings` from env only (12-factor); no literals for secrets/hosts.
- **Structured logging** (structlog, JSON) with a request/trace id bound per request; never log PII or raw tokens.
- **Dependency injection** via `Depends` for db session, current user, services — enables test overrides.

5.2 TypeScript / Next.js

- **TS strict**; **ESLint** + **Prettier**; no `any` without justification.
- **Feature-based folders**; server components for reads, client components only where interactivity requires.
- **Zod** for any client-side form validation; **TanStack Query** for server state.
- Generated API types are the source of truth — do not hand-write response types.

5.3 Testing

- **Backend**: `pytest` + `pytest-asyncio`; unit tests for services (repositories mocked), integration tests against ephemeral Postgres via **testcontainers**; factory fixtures for data. Coverage gate (e.g. ≥80% on services).
- **Frontend**: component tests (Vitest/RTL) + a couple of Playwright E2E flows (capture → space → summary).

- **Contract:** the OpenAPI schema is diffed in CI; breaking changes fail the build.

5.4 Git & CI

- Trunk-based with short-lived branches; **conventional commits**; PRs require green CI + one review.
 - **CI stages:** ruff → mypy → pytest (with services) → build; frontend: eslint → tsc → test → build.
 - **Migrations:** every schema change ships an Alembic migration; no manual DB edits.
-

6. Non-functional requirements

Attribute	Target (POC)
Capture ack latency	p95 < 300 ms (persist + enqueue only)
Summary availability	p95 < 30 s after capture
Chat response	p95 < 5 s
Availability	99.5% (API), queue durable across restarts
Data durability	S3 (11 nines) for raw; nightly Postgres backups
Scale (POC)	100s of concurrent users, low-thousands sources/day

7. Deployment & infrastructure

- **Frontend:** Vercel (Next.js).
- **API + workers:** containerized FastAPI and Arq worker as separate services (Fly.io / Render / ECS). Same image, different entrypoints.
- **Postgres + pgvector:** managed (Supabase / Neon / RDS).
- **Redis:** managed (Upstash / ElastiCache) for the queue.
- **Object store:** R2 or S3.
- **Secrets:** platform secret manager; injected as env; never in the repo.

8. Security considerations

- Cookie flags `Secure; HttpOnly; SameSite=None`; opaque tokens, hashed at rest, with expiry + rotation.

- Authorization checks on every resource (`assert_owned`); never trust client-supplied `user_id`.
- Input validation at the boundary (Pydantic); output encoding on the frontend.
- Rate limiting on `POST /sources` and chat; per-user quotas.
- S3 IAM scoped to the artifact bucket; access only via pre-signed URLs bound to the user prefix.
- CSRF protection for web; `Origin` allow-list for the extension.
- Dependency scanning + secret scanning in CI.

9. Observability

- **Logs:** structured JSON, request/trace id, per-pipeline-stage timing.
- **Metrics:** capture rate, queue depth, stage durations, LLM/embedding cost per source, failure rate by stage.
- **Tracing:** OpenTelemetry across API → queue → worker.
- **Errors:** Sentry on API, workers, and frontend.
- **Alerts:** queue depth, `failed` rate, summary-latency SLO breach.

10. Risks & future work

Risk	Mitigation
AI-chat DOM scraping is fragile	Ship after core capture; per-service content scripts + monitoring.
Concept resolution accuracy	Tune thresholds on a labeled set; advisory lock; adjudication fallback.
LLM cost at scale	Model split, embedding cache, batch calls, cost metrics per user.
pgvector at scale	HNSW now; migrate to dedicated vector DB only on measured strain.
Copyright exposure	Never resurface verbatim source text; summaries + highlights + links only.

Future: mobile retention layer, spaced-repetition scheduler (FSRS) using `next_review_at`, real knowledge-tracing model, syllabus-based coverage.

11. Appendix — glossary

See §1.3. All identifiers are UUIDv4. All times are UTC `timestamptz`. Vector dimension shown as `1536` — set to match the chosen embedding model.